

Project Report: Intergalactic Cargo Portal

Repo Link: <https://github.com/SBiswal02/IntergalacticCargoPortal-Biswal>

Introduction

Intergalactic Cargo Portal is a full-stack cargo manifest management system built with a Node.js/Express backend and a React/Vite frontend. The project implements authentication, backend-controlled role assignment, role-based access control, manifest upload, cargo parsing, database persistence, and a dashboard that changes behavior based on whether the logged-in user is an Admin or a Standard user.

The work can be divided into three major task sections:

- Task 1: Authentication and database schema setup.
- Task 2: Manifest upload, cargo parsing, and database population.
- Task 3: Complete portal experience with role-specific frontend behavior.

Technology Stack

Backend

- Node.js
- Express.js
- SQLite
- Sequelize ORM
- JSON Web Tokens (JWT)
- bcryptjs
- multer

Frontend

- React 18
- Vite
- React Router
- CSS

Testing

- Node.js assert module
- Custom backend test scripts

Task 1: Auth Ready

Submission Body

For Task 1, the required submission body is:

JSON payload a client would send to the /signup endpoint to successfully trigger the creation of an Admin user:

```
{
  "name": "Nova Admin",
  "email": "nova@nebula-corp.com",
  "password": "password123"
}
```

This payload creates an Admin user because the backend checks the submitted email address and automatically assigns the Admin role when the email ends with `@nebula-corp.com`.

Database Schema text snippet:

```
users (  
  id INTEGER PRIMARY KEY,  
  name TEXT,  
  email TEXT UNIQUE,  
  password TEXT,  
  role TEXT CHECK (role IN ('Admin', 'Standard')),  
  created_at TEXT,  
  updated_at TEXT  
)
```

Objective

The objective of Task 1 was to implement a secure authentication foundation for the portal. This included signup, login, password hashing, JWT generation, database-backed user storage, and automatic role assignment.

Implementation Summary

The backend provides `/signup` and `/login` endpoints. A user can create an account by submitting their name, email, and password. During signup, the backend validates the request, normalizes the email address, checks for duplicate users, hashes the password with `bcrypt`, assigns a role, stores the user in the database, and returns a JWT.

The role assignment is handled only by the backend. The client cannot choose whether a user is Admin or Standard. This is important because user roles are security-sensitive and should not be trusted from frontend input.

Admin Role Creation Rule

Users whose email addresses end with `@nebula-corp.com` are assigned the Admin role. All other users are assigned the Standard role.

Example Admin signup payload:

```
{
  "name": "Nova Admin",
  "email": "nova@nebula-corp.com",
  "password": "password123"
}
```

This payload successfully creates an Admin user because the email ends with the approved Admin domain.

Example Standard user:

```
{
  "name": "Orbit User",
  "email": "orbit@example.com",
  "password": "password123"
}
```

This user becomes Standard because the email does not end with `@nebula-corp.com`.

Security Details

The signup endpoint ignores any client-supplied `role` value. Even if a malicious user sends `"role": "Admin"` in the request body, the backend still determines the role from the email domain.

The backend also protects against similar-looking domains. For example, `ops@nebula-corp.com.example` does not become Admin because the email does not truly end with `@nebula-corp.com`.

Passwords are stored as bcrypt hashes, not plain text. After successful signup or login, the backend signs a JWT containing user identity data such as id, name, email, and role.

Database Schema

The authentication feature uses the `users` table.

```
users (  
  id INTEGER PRIMARY KEY,  
  name TEXT,  
  email TEXT UNIQUE,  
  password TEXT,  
  role TEXT CHECK (role IN ('Admin', 'Standard')),  
  created_at TEXT,  
  updated_at TEXT  
)
```

Task 1 Files

- `backend/routes/authRoutes.js` : Signup, login, and current-user endpoint logic.
- `backend/models/User.js` : Sequelize user model.
- `backend/utils/roles.js` : Role resolution based on email domain.
- `backend/tests/auth.test.js` : Authentication and role-assignment tests.

Task 1 Verification

The authentication test verifies that:

- `@nebula-corp.com` users become Admin.
- Non-Nebula users become Standard.
- Client-supplied role values are ignored.
- Similar-looking domains are not treated as Admin domains.
- Login returns a valid token and user role.

Command:

```
cd backend  
npm run test:auth
```

Result:

```
Auth tests passed
```

Task 2: DB Populated

Submission Body

Subject: Update: Task 2 - Biswal - DB Populated

For Task 2, the required submission body is:

Exact cURL command a Standard user would maliciously use to try and hit the upload endpoint:

```
curl -X POST http://localhost:3001/api/upload \  
  -H "Authorization: Bearer dummy-standard-user-token" \  
  -F "manifest=@manifest.txt"
```

Expected error response:

```
{  
  "error": "Clearance level inadequate."  
}
```

The expected HTTP status code is `403 Forbidden` because the upload endpoint is protected by Admin-only role middleware. A Standard user token cannot successfully upload a manifest.

Screenshot attachment:

Task 2 - Screenshot 1 - Biswal - Error

Objective

The objective of Task 2 was to implement the cargo manifest upload workflow, process uploaded cargo records according to the required business rules, save valid cargo entries to the database, and block Standard users from accessing the upload endpoint.

Upload Endpoint

The backend exposes an Admin-only upload endpoint:

```
POST /api/upload
```

The endpoint expects a file field named `manifest`. The uploaded file is handled by `multer` and stored temporarily in the backend uploads directory. After upload, the backend reads the file, parses each manifest line, applies processing rules, saves valid records, and returns import statistics.

Standard User 403 Behavior

Standard users are not allowed to upload manifests. If a Standard user attempts to call the upload endpoint, the request is rejected by role-based middleware.

Example malicious cURL command:

```
curl -X POST http://localhost:3001/api/upload \
  -H "Authorization: Bearer dummy-standard-user-token" \
  -F "manifest=@manifest.txt"
```

Expected response:

```
{
  "error": "Clearance level inadequate."
}
```

The response status is `403 Forbidden`. This confirms that upload access is enforced by the backend, not just hidden in the frontend.

Manifest Format

The parser expects cargo manifest lines in this format:

```
[YYYY-MM-DD] || CRG-001 :: 500 >> Mars Base Alpha
```

Each valid line contains:

- Manifest date
- Cargo code
- Weight
- Destination

Invalid or blank lines are ignored.

Cargo Processing Rules

The parser applies the following rules:

- If the destination contains `Sector-7` , the cargo weight is multiplied by `1.45` .
- The adjusted weight is rounded to the nearest integer.
- If the final weight is a prime number, the cargo record is skipped.
- If the final weight is not prime, the cargo record is saved.

Example:

```
[2026-03-29] || CRG-005 :: 20 >> Sector-7 Mining Rig
```

Processing:

```
20 * 1.45 = 29
```

Because `29` is prime, this record is skipped.

Another example:

```
[2026-03-29] || CRG-012 :: 100 >> Sector-7 Command Center
```

Processing:

```
100 * 1.45 = 145
```

Because `145` is not prime, this record is saved.

Cargo Database Schema

The processed cargo records are stored in the `cargo` table.

```
cargo (  
  id INTEGER PRIMARY KEY,  
  cargo_code TEXT,  
  manifest_date TEXT,  
  weight_kg INTEGER,  
  destination TEXT,  
  created_at TEXT  
)
```

Cargo API

Authenticated users can retrieve saved cargo records using:

```
GET /api/cargo
```

The backend returns records ordered by weight in descending order. The frontend applies one additional display rule by pinning Earth-bound shipments to the bottom of the table.

Task 2 Files

- `backend/routes/cargoRoutes.js` : Upload and cargo listing endpoints.
- `backend/middleware/authMiddleware.js` : JWT authentication middleware.
- `backend/middleware/roleMiddleware.js` : Admin-only route protection.
- `backend/models/Cargo.js` : Sequelize cargo model.
- `backend/utils/parser.js` : Manifest parsing and weight-processing rules.
- `backend/tests/parser.test.js` : Parser behavior tests.
- `manifest.txt` : Sample manifest file.

Task 2 Verification

The parser test verifies that:

- Sector-7 weights are multiplied by `1.45` .
- Adjusted weights are rounded.
- Prime final weights are skipped.
- Non-prime final weights are saved.

Command:

```
cd backend
npm run test:parser
```

Result:

```
Task 2 parser tests passed
```

Task 3: Portal Complete

Objective

The objective of Task 3 was to complete the user-facing portal experience. This included login, signup, logout, role-aware dashboard behavior, Admin upload visibility, Standard user restrictions, weight display differences, and cargo table ordering.

Frontend Authentication Flow

The frontend provides an authentication page with two modes:

- Login
- Sign Up

When a user signs up or logs in successfully, the backend returns a JWT and user object. The frontend stores this authentication state using `AuthContext` . The authenticated user is then shown the dashboard.

The signup form allows users to create new accounts. The UI explains that `@nebula-corp.com` email addresses receive Admin clearance automatically, while the backend remains the actual source of truth for role assignment.

Admin Dashboard Behavior

When an Admin user logs in, the dashboard displays:

- User role badge showing Admin.
- User email.
- Logout button.
- Manifest upload section.
- Cargo shipment table.
- Weights displayed in kilograms as `KG`.

Admin users can upload a manifest file from the dashboard. After upload, the frontend displays a result message showing how many records were imported and skipped. It then reloads the cargo records from the backend.

Standard Dashboard Behavior

When a Standard user logs in, the dashboard displays:

- User role badge showing Standard.
- User email.
- Logout button.
- Cargo shipment table.
- Weights displayed in pounds as `LBS`.

The Standard user does not see the upload panel. This improves the user experience by hiding actions they cannot perform. The backend still enforces the real security restriction, so even a direct API request from a Standard user is blocked.

Weight Display Rules

Cargo weights are stored in the database as kilograms. The frontend displays them differently based on role:

- Admin users see weights in kilograms.
- Standard users see weights converted to pounds.

The conversion uses:

$$1 \text{ KG} = 2.20462 \text{ LBS}$$

The final pound value is rounded for display.

Earth Shipment Ordering

The cargo table includes a special display rule for Earth-bound shipments. Even though other cargo records are sorted by weight, shipments with destination `Earth` are pinned to the bottom of the table.

This behavior is implemented in the frontend cargo utility function. Non-Earth records are sorted by descending weight first, then Earth records are appended at the bottom.

Screen Recording Requirements

The final screen recording should demonstrate:

- Creating a new user.
- Logging in as Admin.
- Admin dashboard showing the upload option.
- Admin dashboard showing weights in KG.
- Logging out.
- Logging in as a Standard user.
- Standard dashboard showing no upload option.
- Standard dashboard showing weights in LBS.
- Earth shipment pinned to the bottom of the table.

Repository Link

<https://github.com/SBiswal02/cargoPortal>

Task 3 Files

- `frontend/src/pages/AuthPage.jsx` : Login and signup page.
- `frontend/src/context/AuthContext.jsx` : Authentication state handling.
- `frontend/src/pages/Dashboard.jsx` : Role-aware dashboard.
- `frontend/src/components/CargoTable.jsx` : Cargo table rendering.
- `frontend/src/utils/cargo.js` : Weight formatting and cargo sorting.
- `frontend/src/api.js` : Frontend API calls.

API Summary

Endpoint	Method	Access	Description
<code>/health</code>	GET	Public	Checks API health
<code>/signup</code>	POST	Public	Creates a user and returns JWT
<code>/login</code>	POST	Public	Authenticates a user and returns JWT
<code>/me</code>	GET	Authenticated	Returns current token user data
<code>/api/upload</code>	POST	Admin	Uploads and processes manifest file
<code>/api/cargo</code>	GET	Authenticated	Returns saved cargo records

Installation and Execution

Backend

```
cd backend
npm install
npm run dev
```

The backend runs at:

```
http://localhost:3001
```

Frontend

```
cd frontend  
npm install  
npm run dev
```

The frontend runs at:

```
http://localhost:5173
```

Project Structure

```
backend/  
  app.js  
  server.js  
  config/  
  middleware/  
  models/  
  routes/  
  tests/  
  uploads/  
  utils/  
  
frontend/  
  index.html  
  src/  
    api.js  
    App.jsx  
    components/  
    context/  
    pages/  
    utils/  
  
manifest.txt
```

README.md

PROJECT_REPORT.md

SUBMISSION_SECTIONS.md

Limitations

- SQLite is appropriate for local development, but a production deployment may require PostgreSQL or MySQL.
- Uploaded manifest files are stored locally, so production use would need file cleanup or cloud storage.
- The current role system supports only Admin and Standard users.
- Backend tests cover authentication and parser behavior, but frontend tests could be added.
- Duplicate cargo detection could be improved if repeated manifest uploads should be prevented.

Future Enhancements

- Add search, filtering, and pagination for large cargo datasets.
- Add upload history for Admin users.
- Add audit logs for security-sensitive actions.
- Add CSV or PDF export for cargo reports.
- Add frontend automated tests.
- Add production-ready environment configuration.
- Add duplicate cargo detection and validation.

Conclusion

Intergalactic Cargo Portal successfully implements all three project stages. Task 1 establishes secure authentication and backend-controlled role assignment. Task 2 adds manifest upload, cargo parsing, database population, and Admin-only upload protection. Task 3 completes the user-facing portal with role-specific dashboard behavior, unit-based weight display, logout/login flow, new user creation, and Earth shipment ordering.

The project demonstrates a practical full-stack implementation using Express, Sequelize, SQLite, React, and Vite. Security-sensitive decisions such as role assignment and upload authorization are enforced on the backend, while the frontend provides a clear and role-aware user experience.